

Superscalar Architecture

Week 7: Debugging, IPC Measurement, and Quartus Synthesis

July 4, 2026

Contents

Week 7 in 60 Seconds	4
1 Week 7 — Measurement, Analysis, and Synthesis	6
2 Comprehensive Theory 1: Measuring IPC [~30 min]	8
2.1 What IPC Is	8
2.2 Why We Count Committed (Not Dispatched or Executed)	8
2.3 Why We Count Cycles From First Fetch to Last Commit	8
2.4 Implementing the Counter Harness	9
2.5 A Worked IPC Calculation	10
3 Comprehensive Theory 2: Understanding the IPC Gap [~40 min]	11
3.1 The Ideal and the Reality	11
3.2 Cause 1: Data Dependencies	11
3.3 Cause 2: Structural Stalls (RS and ROB Full)	11
3.4 Cause 3: The Single Commit Port (Our Specific Limitation)	12
3.5 Cause 4: Wakeup-to-Issue Latency	12
3.6 Cause 5: Pipeline Fill and Drain	12
3.7 Putting It Together: Attributing the Gap	12
3.8 Amdahl’s Law Connection	13
4 Comprehensive Theory 3: Quartus Synthesis Flow [~30 min]	14
4.1 What Synthesis Does	14
4.2 Setting Up the Quartus Project	14
4.3 Reading the Fmax Report	15
4.4 Reading the Critical Path	15
4.5 Reading Resource Utilization	15
4.6 Why Synthesis Can Reveal Bugs Simulation Missed	16
5 Comprehensive Theory 4: Synthesis-Specific Bugs [~20 min]	17
5.1 Bug 1: Inferred Latches	17
5.2 Bug 2: Combinational Loops	17
5.3 Bug 3: Multiple Drivers	18
5.4 Bug 4: Unsupported Constructs	18
5.5 Bug 5: Width Mismatches	18
5.6 Bug 6: Don’t-Care Propagation	18
5.7 The Synthesis Workflow	18
6 Optional Stretch Goals: MUL and LSU [variable time]	20
6.1 The Multiplier (MUL)	20
6.2 The Load-Store Unit (LSU)	20
6.3 If You Add These, Re-Measure	21

6.4	Honest Advice	21
7	The Benchmarks and the Performance Report [~2 hours]	22
7.1	Benchmark 1: Independent	22
7.2	Benchmark 2: Dependency Chain	22
7.3	Benchmark 3: Mixed	23
7.4	The Performance Report (<code>performance.md</code>)	23
7.5	Example Gap-Analysis Paragraph (For Reference)	24
8	Week 7 Deliverables	25
A	Appendix: Solutions to Concept Checks	27

Week 7 in 60 Seconds

A one-page summary. This week, the question changes from “*does it work?*” to “*how well does it work, and why isn’t it faster?*” No major new modules are required (though MUL and LSU are available as stretch goals). The work is measurement, analysis, and synthesis.

The conceptual shift: For six weeks you’ve been chasing correctness. This week you measure performance and confront the gap between your design’s theoretical peak ($IPC = 2.0$) and what it actually achieves (likely 1.1–1.5). That gap is not a bug — it is the entire field of computer architecture, visible in your own design.

IPC — the key metric:

$$IPC = \frac{\text{instructions committed}}{\text{total cycles}}$$

A perfect 2-wide machine on perfectly parallel code achieves $IPC = 2.0$. Your design will fall short, and your job this week is to explain *why* — attributing the gap to specific causes.

The sources of the IPC gap (you’ll measure these):

- **Dependency chains:** when each instruction depends on the previous, no parallelism is available — IPC drops toward 1.
- **RS / ROB structural stalls:** when these fill up, dispatch stalls.
- **Single commit port (our limitation):** we can only return one freed physical register per cycle, which can throttle the free list.
- **Wakeup-to-issue latency:** the 1-cycle producer-to-consumer delay.
- **Pipeline fill/drain:** the first few and last few cycles of any program are not at full throughput.

Three benchmark programs you’ll run:

1. **Independent** — 16 instructions with no dependencies. Should approach $IPC = 2.0$.
2. **Dependency chain** — each instruction depends on the previous. Should approach $IPC = 1.0$.
3. **Mixed** — realistic blend. Somewhere in between.

Measuring all three tells you the *range* of your machine’s behavior.

Quartus synthesis: You’ll synthesize PRAVAH for an Intel Cyclone V / DE10-Lite target. The synthesis produces three things you’ll report:

- **Fmax:** maximum clock frequency (limited by the critical path).
- **Logic utilization:** how many logic elements (LEs / ALMs) the design uses.
- **Critical path:** the slowest combinational path, which limits Fmax. For PRAVAH, this is usually the wakeup-select loop.

You will *not* deploy on a physical board (unless you have one and want to). Synthesis-to-report is the requirement.

This week's deliverable: A 1–2 page `performance.md` report containing IPC for all 3 benchmarks, a Quartus timing/resource summary, and a written analysis attributing the IPC gap to specific causes.

Suggested reading order: Section 1 covers IPC measurement methodology. Section 2 is the gap analysis (the deep part). Section 3 is the Quartus synthesis flow. Section 4 is synthesis-specific bugs. Section 5 is optional MUL/LSU stretch goals. Section 6 is deliverables.

1 Week 7 — Measurement, Analysis, and Synthesis

Goal

Quantify PRAVAH's performance and understand its limits. Measure IPC on three benchmarks, synthesize on Quartus, extract Fmax and resource usage, and write a performance report that attributes the IPC gap to specific architectural causes. Optionally extend the design with a multiplier and/or load-store unit.

Mentee Work (10 hours)

- Read Sections 1–2 (IPC methodology and gap analysis, ~1.5 hours).
- Build the cycle-counting harness in the testbench (~1 hour).
- Write 3 benchmark programs and measure IPC (~2 hours).
- Read Section 3 (Quartus flow, ~45 min). Run synthesis and extract reports (~2 hours, mostly fighting the toolchain).
- Read Section 4 (synthesis bugs) and fix any that arise (~1 hour).
- Write `performance.md` (~1.5 hours).
- *Optional*: implement MUL and/or LSU (Section 5).

Deliverables

- `programs/bench_independent.hex`, `bench_chain.hex`, `bench_mixed.hex` — the three benchmarks.
- `tb/tb_pravah_perf.v` — testbench with cycle-counting harness.
- `docs/performance.md` — 1–2 page report (IPC table, Quartus summary, gap analysis).
- `docs/week7/quartus_timing.png`, `quartus_resources.png` — synthesis report screenshots.
- Updated `learning_log.md`.

What Mentor Will Look For

- IPC measured correctly for all 3 benchmarks (independent near 2.0, chain near 1.0, mixed in between).
- Quartus synthesis completes; Fmax and LE usage reported.

- Critical path identified (and ideally it's the wakeup-select loop — if it's something else, that's interesting and worth discussing).
- Gap analysis is specific: not “the IPC is lower because of overhead” but “the chain benchmark achieves IPC 0.95 because every instruction depends on the previous, leaving no parallelism for the second issue slot.”

2 Comprehensive Theory 1: Measuring IPC [~30 min]

2.1 What IPC Is

IPC (Instructions Per Cycle) is the average number of instructions that complete per clock cycle:

$$\text{IPC} = \frac{\text{number of instructions committed}}{\text{number of clock cycles}}$$

It is the inverse of CPI (Cycles Per Instruction), which you met in Week 1. We use IPC rather than CPI for superscalar machines because $\text{IPC} > 1$ is the whole point — and “IPC = 1.8” is more intuitive than “CPI = 0.56.”

2.2 Why We Count Committed (Not Dispatched or Executed)

There are several points where you could count instructions:

- **Fetch:** includes speculative instructions that may be squashed. Overcounts.
- **Dispatched:** includes instructions that haven’t finished. Overcounts in-flight work.
- **Committed:** only counts instructions that actually retired and affected architectural state. **This is the correct count.**

We count committed instructions because IPC is meant to measure *useful work completed*, and an instruction only counts as “done” when it commits. Speculative instructions that get squashed represent wasted work, correctly excluded.

2.3 Why We Count Cycles From First Fetch to Last Commit

The cycle count should span from the first instruction’s fetch to the last instruction’s commit. This includes:

- **Pipeline fill:** the first few cycles where the pipeline is partially empty (instructions are working their way through fetch/decode/rename before any commit).
- **Steady state:** the bulk of execution at (hopefully) high IPC.
- **Pipeline drain:** the last few cycles where the last instructions are committing but no new ones are being fetched.

For a short benchmark (16 instructions), the fill and drain are a significant fraction of total cycles, which artificially lowers IPC. For longer benchmarks, fill and drain amortize away. We’ll account for this in the analysis.

2.4 Implementing the Counter Harness

The cycle-counting harness lives in the testbench (not the RTL — it's measurement, not hardware). It does two things:

1. Count total cycles from reset-release to the cycle the last instruction commits.
2. Count total committed instructions.

```
1 // Inside tb_pravah_perf.v
2
3 integer cycle_count = 0;
4 integer commit_count = 0;
5 integer first_commit_cycle = -1;
6 integer last_commit_cycle = 0;
7
8 // Count cycles after reset
9 always @(posedge clk) begin
10     if (~reset)
11         cycle_count <= cycle_count + 1;
12 end
13
14 // Count commits (peek into the ROB's commit signals)
15 always @(posedge clk) begin
16     if (~reset) begin
17         if (dut.u_rob.commit_A_valid_o) begin
18             commit_count <= commit_count + 1;
19             if (first_commit_cycle < 0) first_commit_cycle <=
20                 cycle_count;
21             last_commit_cycle <= cycle_count;
22         end
23         if (dut.u_rob.commit_B_valid_o) begin
24             commit_count <= commit_count + 1;
25             last_commit_cycle <= cycle_count;
26         end
27     end
28 end
29
30 // At end of simulation, report
31 initial begin
32     #5000; // run long enough
33     $display("=== Performance Report ===");
34     $display("Instructions committed: %0d", commit_count);
35     $display("Total cycles (reset to last commit): %0d",
36         last_commit_cycle);
37     $display("IPC = %f", commit_count * 1.0 / last_commit_cycle);
```

```
36     $finish;  
37 end
```

Note: we peek into the DUT’s internal commit signals (`dut.u_rob.commit_A_valid_o`) from the testbench. This is fine for a testbench — it’s a non-synthesizable observation. In a real chip you’d add a performance counter in hardware, but for our purposes the testbench peek is simpler.

2.5 A Worked IPC Calculation

Suppose a 16-instruction benchmark commits all 16 instructions, and the last commit happens at cycle 20 (counting from reset release).

$$\text{IPC} = \frac{16}{20} = 0.80$$

Wait — that’s less than 1, on a 2-wide machine? Yes, because of pipeline fill and drain. The first commit might not happen until cycle 6 (after the first instruction works through fetch, decode, rename, dispatch, execute, commit). So we have 16 instructions committing over cycles 6–20 = 14 cycles of “active commit,” but the cycle count from reset is 20.

Two ways to report IPC:

- **End-to-end IPC:** $16 / 20 = 0.80$. Includes fill/drain. This is the honest, conservative number.
- **Steady-state IPC:** $16 / (20 - 6 + 1) = 16/15 \approx 1.07$. Excludes fill. More representative of a long-running program.

Report both in your `performance.md`, and explain the difference. For short benchmarks, the gap between them is large; understanding why is part of the learning.

Concept Check 1

1. Why do we count committed instructions rather than fetched or dispatched?
2. A benchmark commits 32 instructions; the first commit is at cycle 5 and the last at cycle 25. Compute end-to-end IPC and steady-state IPC.
3. Why is the gap between end-to-end and steady-state IPC larger for short benchmarks than long ones?

Going deeper: Hennessy & Patterson §1.9 discusses performance measurement principles. The “Iron Law” from Week 1 is the foundation: $\text{execution time} = \text{instructions} \times \text{CPI} \times \text{cycle time}$. $\text{IPC} = 1/\text{CPI}$.

3 Comprehensive Theory 2: Understanding the IPC Gap [~40 min]

This is the conceptual heart of Week 7. Your machine has an ideal IPC of 2.0, but you'll measure something lower. Understanding *why* is the deepest lesson of the entire program.

3.1 The Ideal and the Reality

PRAVAH can fetch, dispatch, and commit up to 2 instructions per cycle. So its theoretical ceiling is $\text{IPC} = 2.0$. You'll never reach it on real code. The gap comes from a handful of specific causes, each of which you can measure and reason about.

3.2 Cause 1: Data Dependencies

This is the dominant cause and the most fundamental. If instruction B needs the result of instruction A, B cannot execute until A finishes. No amount of hardware parallelism helps — the dependency is in the program, not the machine.

Extreme case: a pure dependency chain.

```
add x1, x0, x0      # x1 = 0
add x2, x1, x1      # depends on x1
add x3, x2, x2      # depends on x2
add x4, x3, x3      # depends on x3
...
```

Each instruction depends on the previous. The second issue slot is useless — there's never a second independent instruction to issue. **IPC approaches 1.0**, regardless of how wide the machine is. This is your `bench_chain` benchmark.

Opposite extreme: fully independent instructions.

```
add x1, x0, x0
add x2, x0, x0
add x3, x0, x0
add x4, x0, x0
...
```

Every instruction is independent. Both issue slots stay busy. **IPC approaches 2.0** (minus fill/drain). This is your `bench_independent` benchmark.

Real code is between these extremes. The “available instruction-level parallelism” (ILP) of a program is an intrinsic property; the machine can only exploit ILP that exists.

3.3 Cause 2: Structural Stalls (RS and ROB Full)

Even with available ILP, the machine has finite resources:

- **4 ALU reservation stations.** If more than 4 ALU instructions are waiting, dispatch stalls.

- **8 ROB entries.** If 8 instructions are in flight (dispatched but not committed), dispatch stalls until something commits.
- **16 free physical registers.** If all are allocated, dispatch stalls.

These stalls manifest when a long-latency instruction (or a dependency chain) keeps the ROB head occupied, preventing commit, which prevents freeing resources, which eventually stalls dispatch. The ROB size is often the binding constraint.

3.4 Cause 3: The Single Commit Port (Our Specific Limitation)

Recall from Week 6: PRAVAH's rename unit has only *one* commit port. So even though the ROB can identify 2 committable instructions per cycle, only one physical register can be returned to the free list per cycle.

Effect: in a workload that commits 2 instructions per cycle for a while, the free list drains faster than it refills, eventually stalling dispatch. This is a self-inflicted limitation we chose for simplicity.

Measuring it: compare the IPC of `bench_independent` to the theoretical 2.0. If it's notably below 2.0 even accounting for fill/drain, the single commit port is likely a contributor. (Widening to 2 commit ports is a stretch goal — if you do it, re-measure and see the IPC improve.)

3.5 Cause 4: Wakeup-to-Issue Latency

Recall the 1-cycle producer-to-consumer latency from Week 6: an instruction that finishes in cycle T can only wake its consumer in cycle $T+1$. In a dependency chain, this 1-cycle latency per link is unavoidable in our design.

Effect on the chain benchmark: each dependent instruction waits 1 cycle after its producer. For a chain of N instructions, that's roughly N cycles minimum — $IPC = 1.0$. (With early wakeup, you could approach the same chain in fewer cycles, but we don't implement it.)

3.6 Cause 5: Pipeline Fill and Drain

At the start, the pipeline is empty — it takes several cycles before the first instruction commits. At the end, the last instructions drain out while no new ones enter. For a 16-instruction benchmark with a 5-stage front-end, fill + drain might be 8–10 cycles — a large fraction of total runtime.

Effect: short benchmarks have artificially low end-to-end IPC. This is why we report steady-state IPC separately and why real benchmarks are millions of instructions long.

3.7 Putting It Together: Attributing the Gap

A good performance analysis sounds like this:

“The independent benchmark achieved end-to-end IPC of 1.6 and steady-state IPC of 1.9. The shortfall from the 2.0 ceiling in steady state is attributable primarily to the single commit port, which throttles free-register return when both slots want to commit. The chain benchmark achieved IPC 0.95, essentially the $IPC=1.0$ floor expected for a fully serial dependency chain, with the slight shortfall due to the 1-cycle wakeup-to-issue latency between each dependent pair. The mixed benchmark achieved IPC 1.35, reflecting partial ILP exploitation.”

Notice: every number is attributed to a *specific* cause. That’s what we want. “Overhead” and “various factors” are not acceptable; name the mechanism.

3.8 Amdahl’s Law Connection

Week 1 introduced the Iron Law. Amdahl’s Law is its cousin for parallelism: the speedup from parallelism is limited by the serial fraction of the work. In OoO terms: the speedup from a wide machine is limited by the dependency chains in the code (the serial fraction). A program that is 50% dependency-chain and 50% independent can never exceed an average IPC determined by that mix, no matter how wide the machine.

This is why processor architects care so much about ILP extraction (and why compilers reorder code to expose more parallelism). The hardware can only run as fast as the program’s dependency structure allows.

Concept Check 2

1. Why does a pure dependency chain achieve $IPC = 1.0$ regardless of issue width?
2. Your `bench_independent` measures steady-state $IPC = 1.7$, below the 2.0 ceiling. Name two specific causes you’d investigate.
3. If you widened the commit port to 2 and re-measured `bench_independent`, would you expect the chain benchmark’s IPC to change? Why or why not?

Going deeper: Hennessy & Patterson §3.1 discusses ILP limits. The classic “Limits of Instruction-Level Parallelism” study by David Wall (1991) measured the available ILP in real programs — a fascinating read if you have time.

4 Comprehensive Theory 3: Quartus Synthesis Flow

[~30 min]

So far, everything has run in simulation (ModelSim). Simulation verifies functional correctness but tells you nothing about whether the design could run on real hardware, how fast, or how big. Synthesis answers those questions.

4.1 What Synthesis Does

Synthesis translates your Verilog RTL into a netlist of physical FPGA primitives: lookup tables (LUTs), flip-flops, and the routing between them. Quartus then:

1. **Analysis & Synthesis:** parses your Verilog, infers logic, maps to FPGA primitives.
2. **Fitter (Place & Route):** assigns each primitive to a physical location on the FPGA and routes the wires.
3. **Timing Analysis:** computes the maximum clock frequency based on the longest combinational path.
4. **Assembler:** generates the bitstream (only needed if deploying to a board).

For PRAVAH, we run through Timing Analysis and read the reports. We don't need the bitstream unless you have a physical board.

4.2 Setting Up the Quartus Project

1. Create a new Quartus project. Target device: Cyclone V (e.g., 5CSEMA5F31C6, the DE10-Standard chip) or MAX 10 (10M50DAF484C7G, the DE10-Lite chip). Either is fine; pick whichever matches a board you might have, or just pick DE10-Lite as the default.
2. Add all your `rtl/*.v` files to the project. **Do not add testbenches** — they're not synthesizable.
3. Set `pravah_top` as the top-level entity.
4. Create a timing constraint: tell Quartus your target clock. Create an `.sdc` file with: `create_clock -period 20 [get_ports clk_i]` (20 ns = 50 MHz target).
5. Run "Analysis & Synthesis" (the first stage). Fix any errors (Section 4 covers common ones).
6. Run the full compilation (Analysis & Synthesis + Fitter + Timing Analysis).

4.3 Reading the Fmax Report

After compilation, open the Timing Analyzer (TimeQuest). The key number is **Fmax** — the maximum frequency at which your design will work correctly.

- Look for the “Slow 1100mV 85C Model” Fmax (the worst-case corner).
- This is your design’s speed limit. If it’s, say, 65 MHz, your design works up to 65 MHz; beyond that, the critical path can’t settle in one clock period.
- For PRAVAH, expect Fmax somewhere in the 40–80 MHz range, depending on how your wakeup-select logic was written. This is normal for a student OoO design — real CPUs achieve GHz frequencies through deep pipelining and aggressive circuit design we don’t attempt.

4.4 Reading the Critical Path

The critical path is the longest combinational path between two registers. It determines Fmax ($F_{\max} = 1 / \text{critical-path-delay}$).

In the Timing Analyzer, look at the “Report Timing” for the worst-case path. For PRAVAH, the critical path is usually one of:

- **The wakeup-select loop:** PRF ready bit → RS wakeup logic → priority encoder → issue mux → ALU → PRF write. This is a long combinational chain. If this is your critical path, your design is “wakeup-select limited” — the same limitation real OoO processors face.
- **The rename bypass logic:** the intra-bundle dependency muxes can form a long path if not carefully structured.
- **The ALU:** for wide operations, but usually not the bottleneck for our simple ALU.

Identifying your critical path is a required part of the performance report. It tells you what to optimize if you wanted to go faster.

4.5 Reading Resource Utilization

The compilation report shows how much of the FPGA your design uses:

- **Logic elements (LEs) or ALMs:** the combinational + sequential logic. PRAVAH should use a few thousand — well within any modern FPGA.
- **Registers:** the flip-flops (PRF, ROB, RS state, pipeline registers).
- **Memory bits:** if any structures were inferred as block RAM.

For PRAVAH, resource usage is not a concern — the design is small. But reporting it is good practice and tells you how the design scales.

4.6 Why Synthesis Can Reveal Bugs Simulation Missed

Simulation and synthesis can disagree. Code that simulates correctly might:

- Synthesize to a latch (unintended) instead of combinational logic.
- Have a combinational loop (a wire that depends on itself), which simulation tolerates but synthesis rejects.
- Use a construct the synthesizer doesn't support.

Section 4 covers these. The lesson: **a design isn't "done" until it both simulates correctly and synthesizes cleanly.**

Concept Check 3

1. What does Fmax represent physically? If your Fmax is 50 MHz, what happens if you clock the design at 80 MHz?
2. Why is the wakeup-select loop a likely critical path for an OoO design?
3. Why don't we need to generate the bitstream (run the Assembler) for this project?

Going deeper: Intel's Quartus Prime documentation has a "Timing Analysis Overview" that explains setup/hold and Fmax computation. The DE10-Lite User Manual (if you have the board) explains the physical I/O.

5 Comprehensive Theory 4: Synthesis-Specific Bugs

[~20 min]

Your design simulated correctly in Weeks 3–6. Synthesis may still complain. Here are the bugs that pass ModelSim but fail (or warn in) Quartus.

5.1 Bug 1: Inferred Latches

Cause: a combinational `always @*` block that doesn't assign a signal in every code path. The synthesizer infers a latch to "hold the previous value."

Symptom: Quartus warning "inferred latch for signal X."

Example:

```
1 // BAD: latch inferred for result when op is not 0 or 1
2 always @* begin
3     case (op)
4         4'd0: result = a + b;
5         4'd1: result = a - b;
6         // no default! result holds its old value otherwise ->
7             latch
8     endcase
9 end
```

Fix: always assign a default at the top of the block, or include a `default` in every case:

```
1 always @* begin
2     result = 32'b0; // default
3     case (op)
4         4'd0: result = a + b;
5         4'd1: result = a - b;
6         default: result = 32'b0;
7     endcase
8 end
```

5.2 Bug 2: Combinational Loops

Cause: a wire whose value depends on itself through combinational logic (no register in the loop).

Symptom: Quartus error "combinational loop detected" or the design oscillates in timing analysis.

Common source in PRAVAH: if you tried to implement "early wakeup" carelessly, you might create a loop where wakeup depends on issue, which depends on wakeup. The proper design always has a register (the PRF ready bit, latched) breaking the loop.

Fix: ensure every feedback path goes through at least one flip-flop. The wakeup-select loop in PRAVAH is broken by the PRF ready bit being a register (latched at the clock edge), not a combinational wire.

5.3 Bug 3: Multiple Drivers

Cause: two `assign` statements or two `always` blocks both driving the same wire.

Symptom: Quartus error “net has multiple drivers” or “X” values in simulation.

Common source: during integration, accidentally connecting two module outputs to the same wire.

Fix: each wire has exactly one driver. If you need to select between sources, use an explicit mux.

5.4 Bug 4: Unsupported Constructs

Cause: using simulation-only constructs in synthesizable RTL.

Symptom: Quartus error on `$display`, `#delay`, `initial` (for logic, not memory init), etc.

Fix: these belong only in testbenches. If you have any in `rtl/`, move them out. The one exception: `initial` blocks for memory initialization (`imem`) are tolerated by Quartus for FPGA block RAM, but it’s cleaner to use `$readmemh` in the testbench and a memory-initialization file (`.mif`) for synthesis.

5.5 Bug 5: Width Mismatches

Cause: assigning a wide signal to a narrow one, or vice versa.

Symptom: Quartus warning “truncated value” or “width mismatch.”

Fix: make widths match explicitly. Use part-selects or zero/sign extension. These are usually warnings, not errors, but they often hide real bugs — investigate each one.

5.6 Bug 6: Don’t-Care Propagation

Cause: simulation initializes registers to X (unknown); synthesis initializes to 0 (or whatever the reset specifies). If your design depends on a register being a specific value before reset, simulation and synthesis diverge.

Fix: make sure every register has a defined reset value, and your design doesn’t depend on pre-reset state. (You should have done this in Weeks 3–6.)

5.7 The Synthesis Workflow

1. Run Analysis & Synthesis only first (faster than full compile).
2. Read every warning. Latches and width mismatches are the dangerous ones.

3. Fix issues, re-run.
4. Once Analysis & Synthesis is clean, run the full compile for timing.
5. Re-verify in ModelSim after any RTL change — synthesis fixes shouldn't break functionality, but verify.

Concept Check 4

1. Why does an incomplete **case** in a combinational block infer a latch? Why is a latch bad?
2. How does the PRF ready bit break the potential combinational loop in the wakeup-select path?
3. Why might a design that simulates perfectly fail synthesis with a “multiple drivers” error?

6 Optional Stretch Goals: MUL and LSU [variable time]

If you finished Milestone 3 comfortably and have time, extend PRAVAH with a multiplier and/or load-store unit. These were deferred from Week 6. They are **not required** — a working ALU-only PRAVAH is a complete project.

6.1 The Multiplier (MUL)

Goal: add a multiply instruction (RV32M’s MUL), executed on a dedicated multi-cycle multiplier with its own reservation stations.

Design:

- Add 2 MUL reservation stations (separate from the 4 ALU RSs).
- The MUL unit takes 3 cycles (pipelined: accept a new MUL each cycle, results emerge 3 cycles later).
- MUL writeback uses a third PRF write port — OR shares the existing 2 ports via arbitration (this is where “the CDB” arbitration actually matters).

The arbitration challenge: now you can have ALU0, ALU1, *and* MUL all finishing in the same cycle. Three writers, two PRF write ports. One must wait. Implement a priority: e.g., MUL has highest priority (it waited longest), then ALU0, then ALU1. The loser holds its result and retries next cycle. This requires the FU to have a “stall” input and hold its output.

Why this is genuinely harder: the FU now needs state (to hold a result that couldn’t write back). The clean combinational ALU becomes a small state machine. Budget real time for this.

6.2 The Load-Store Unit (LSU)

Goal: add LW and SW, executed on a load-store unit that accesses data memory.

Design (simplest version):

- Add 2 LSU reservation stations.
- The LSU computes the effective address (base + offset) in one cycle, then accesses data memory in the next.
- **Loads** write the loaded value to the PRF (like an ALU result).
- **Stores** are special: they must *not* write memory until commit (speculative stores would corrupt memory irreversibly — recall Week 3). So a store sits in the ROB until it reaches the head, and only then writes data memory.

The store-at-commit challenge: this requires the ROB to signal the LSU at commit time for store instructions. The store’s data and address must be held somewhere (a store buffer) between execution and commit. This is a meaningful addition to the ROB.

Memory ordering: we keep it simple — loads block (the LSU handles one memory operation at a time). No load-store queue, no store-to-load forwarding. A load that follows a store to the same address must wait for the store to commit. This is slow but correct.

6.3 If You Add These, Re-Measure

After adding MUL or LSU, re-run your benchmarks (add MUL-heavy and memory-heavy variants) and re-measure IPC. Document how the new units affect performance. A MUL-heavy benchmark will show the 3-cycle MUL latency limiting IPC; a memory-heavy benchmark will show the blocking LSU as the bottleneck.

6.4 Honest Advice

Most mentees should **not** attempt these. The ALU-only PRAVAH is a complete, impressive project. Adding MUL/LSU is genuinely a multi-day effort with new failure modes (FU stalls, store buffers, memory ordering). Only attempt if:

- Milestone 3 is solidly done and tested.
- You have spare time in Week 7 and into Week 8.
- You’re comfortable debugging without it derailing your final report and presentation.

A polished ALU-only project beats a half-working ALU+MUL+LSU project every time.

Concept Check 5

1. Why must stores wait until commit to write memory, while loads can execute (and write the PRF) as soon as their address is ready?
2. Adding a MUL creates a 3-writer / 2-write-port contention. Why didn’t this contention exist in the ALU-only design?
3. Why is a blocking LSU (one memory op at a time) “slow but correct”? What would a load-store queue add?

7 The Benchmarks and the Performance Report [~2 hours]

7.1 Benchmark 1: Independent

16 instructions, no dependencies. Every instruction writes a distinct register and reads only x0 or already-settled registers.

```
addi x1, x0, 1
addi x2, x0, 2
addi x3, x0, 3
addi x4, x0, 4
addi x5, x0, 5
addi x6, x0, 6
addi x7, x0, 7
addi x8, x0, 8
addi x9, x0, 9
addi x10, x0, 10
addi x11, x0, 11
addi x12, x0, 12
addi x13, x0, 13
addi x14, x0, 14
addi x15, x0, 15
addi x16, x0, 16
```

Expected: IPC approaching 2.0 (minus fill/drain and the single commit port).

7.2 Benchmark 2: Dependency Chain

16 instructions, each depending on the previous.

```
addi x1, x0, 1
add x2, x1, x1
add x3, x2, x2
add x4, x3, x3
add x5, x4, x4
add x6, x5, x5
add x7, x6, x6
add x8, x7, x7
add x9, x8, x8
add x10, x9, x9
add x11, x10, x10
add x12, x11, x11
add x13, x12, x12
add x14, x13, x13
add x15, x14, x14
add x16, x15, x15
```

Expected: IPC approaching 1.0 (the serial floor, plus 1-cycle wakeup latency per link).

7.3 Benchmark 3: Mixed

16 instructions, alternating between independent pairs and dependencies.

```
addi x1, x0, 10      # independent
addi x2, x0, 20      # independent
add  x3, x1, x2      # depends on both above
add  x4, x1, x2      # depends on both above (independent of x3)
add  x5, x3, x4      # depends on x3, x4
addi x6, x0, 30      # independent
add  x7, x5, x6      # depends on x5, x6
addi x8, x0, 40      # independent
add  x9, x7, x8      # depends on x7, x8
addi x10, x0, 50     # independent
add  x11, x9, x10    # depends
add  x12, x1, x2     # independent of recent chain
add  x13, x11, x12   # depends
add  x14, x6, x8     # independent of recent chain
add  x15, x13, x14   # depends
add  x16, x15, x1    # depends
```

Expected: IPC between 1.0 and 2.0, reflecting partial ILP.

7.4 The Performance Report (performance.md)

A 1–2 page markdown document containing:

1. **IPC table:** for each benchmark, end-to-end IPC and steady-state IPC.

Benchmark	End-to-end IPC	Steady-state IPC
Independent	(your number)	(your number)
Chain	(your number)	(your number)
Mixed	(your number)	(your number)

2. **Quartus summary:** Fmax, logic-element count, register count, and the identified critical path.
3. **Gap analysis:** 2–3 paragraphs attributing each benchmark’s IPC to specific causes. Use the framework from Section 2. Name mechanisms, not vague “overhead.”
4. **Optional:** if you added MUL/LSU, include MUL-heavy and memory-heavy benchmark results and discuss.

7.5 Example Gap-Analysis Paragraph (For Reference)

The chain benchmark achieved steady-state IPC of 0.93. This is close to the theoretical floor of 1.0 for a fully serial dependency chain: since each instruction depends on its immediate predecessor, the second issue slot is never used, and the machine effectively executes one instruction per cycle. The slight shortfall below 1.0 reflects the 1-cycle wakeup-to-issue latency: after instruction N 's result is written to the PRF in cycle T , instruction $N+1$ cannot issue until cycle $T+1$, introducing a one-cycle bubble per dependency edge. With 15 dependency edges across 16 instructions, this accounts for the observed cycle count.

This is the level of specificity we want.

8 Week 7 Deliverables

In Your Repo

- `programs/bench_independent.hex`, `bench_chain.hex`, `bench_mixed.hex`.
- `tb/tb_pravah_perf.v` — testbench with the cycle-counting harness.
- `docs/performance.md` — the 1–2 page performance report.
- `docs/week7/quartus_timing.png` — Fmax / timing screenshot.
- `docs/week7/quartus_resources.png` — resource utilization screenshot.
- `quartus/` — the Quartus project files (`.qpf`, `.qsf`, `.sdc`).
- *Optional*: `rtl/mul.v`, `rtl/lsu.v` if you did the stretch goals.
- Updated `learning_log.md`.

What Mentor Will Check

1. All 3 benchmarks run and produce IPC numbers in the expected ranges (independent near 2, chain near 1, mixed in between).
2. Quartus synthesis completes; Fmax and resources reported.
3. Critical path identified and named.
4. Gap analysis attributes IPC to specific mechanisms, not vague hand-waving.
5. Design still passes the Milestone 3 correctness test (synthesis fixes didn't break functionality).
6. Plain Verilog only.

Common Issues This Week

- **IPC numbers nonsensical** (e.g., > 2.0 or negative): your cycle counter or commit counter is wrong. Verify the harness against a hand-traced small program.
- **Quartus won't compile** but ModelSim does: work through Section 4's synthesis bugs. Latches and multiple drivers are the usual culprits.
- **Fmax suspiciously high or low**: if Fmax is 500 MHz, you probably didn't set up the timing constraint (`.sdc`). If it's 5 MHz, you may have a pathologically long combinational path — investigate the critical path report.
- **Chain IPC below 1.0 by a lot** (e.g., 0.5): something is adding extra stalls per dependency. Check your wakeup logic isn't introducing more than 1 cycle of latency per link.

Time Estimate

- Reading (Sections 1–4): ~1.5 hours.
- Cycle-counting harness: ~1 hour.
- Writing 3 benchmarks + measuring: ~2 hours.
- Quartus setup + synthesis + report extraction: ~2–3 hours (the toolchain fights you).
- Fixing synthesis bugs: ~1 hour.
- Writing performance.md: ~1.5 hours.

Total: ~9–10 hours for the required work. MUL/LSU stretch goals are extra.

A Appendix: Solutions to Concept Checks

Concept Check 1 (Measuring IPC)

1. Because IPC measures *useful work completed*. Fetched and dispatched instructions include speculative work that may be squashed (wasted) and in-flight work that hasn't finished. Only committed instructions have actually retired and affected architectural state, so committing is the correct “done” point.
2. End-to-end $IPC = 32 / 25 = 1.28$. Steady-state $IPC = 32 / (25 - 5 + 1) = 32 / 21 = 1.52$. (The steady-state excludes the 5-cycle fill before the first commit.)
3. Because fill and drain are roughly constant (set by pipeline depth), while the steady-state portion grows with benchmark length. For a 16-instruction benchmark, a 6-cycle fill is a huge fraction; for a million-instruction benchmark, it's negligible. So the end-to-end and steady-state numbers converge for long benchmarks and diverge for short ones.

Concept Check 2 (The IPC Gap)

1. Because each instruction must wait for its predecessor's result before it can execute. There is never a second independent instruction available to fill the second issue slot. The dependency is inherent in the program; no hardware width can break it. So at most one instruction completes per cycle: $IPC = 1.0$.
2. Two candidates: (a) the single commit port throttling free-register return when both slots want to commit; (b) pipeline fill/drain still affecting even the steady-state estimate if the benchmark is very short. You could also investigate ROB-full stalls or free-list exhaustion.
3. No, the chain benchmark's IPC would not change. The chain is limited by data dependencies, not by the commit port. Widening the commit port helps workloads that commit 2/cycle (like the independent benchmark), but the chain only ever commits 1/cycle anyway (because it only ever finishes 1/cycle). The bottleneck is different.

Concept Check 3 (Quartus Synthesis)

1. Fmax is the maximum clock frequency at which the longest combinational path can settle within one clock period. At 50 MHz Fmax, clocking at 80 MHz means the critical path doesn't finish computing before the next clock edge captures it — you get wrong values (setup violations). The design malfunctions.
2. Because the wakeup-select loop is a long combinational chain: PRF ready bit → wakeup AND-gates → priority encoder → issue mux → ALU compute → writeback.

Many levels of logic in series, no register breaking them within a cycle. This is the same fundamental limitation real OoO processors face, which is why they pipeline it carefully.

3. Because we're only measuring (Fmax, resources, critical path), not deploying. The bitstream (Assembler output) is only needed to actually program a physical FPGA. Timing analysis runs before the Assembler, so we get all our numbers without generating the bitstream.

Concept Check 4 (Synthesis Bugs)

1. An incomplete **case** leaves some input combinations unhandled. In a combinational block, the synthesizer must produce *some* output for every input — so it infers a latch that “holds the previous value” for the unhandled cases. Latches are bad because they're level-sensitive (not edge-triggered), aren't controlled by the clock, create timing-analysis difficulties, and usually indicate a design mistake.
2. The PRF ready bit is a registered value (a flip-flop), updated only at the clock edge. The wakeup logic reads this registered value combinatorially, but the value itself can only change at a clock edge. So the loop “wakeup → issue → execute → writeback → ready bit → wakeup” is broken by the flip-flop on the ready bit — there's a register in the cycle, so it's not a combinational loop.
3. Because in simulation, multiple drivers might resolve to a defined value if only one is actively driving (the others drive Z or are masked). But synthesis sees two physical sources connected to one wire, which is electrically illegal (short circuit if they drive opposite values). Simulation can mask this; synthesis cannot.

Concept Check 5 (MUL/LSU Stretch)

1. Because a store modifies memory *irreversibly*. If a store executed speculatively and then had to be squashed (e.g., a mispredicted branch before it), the memory change couldn't be undone. By waiting until commit, we guarantee the store is non-speculative. A load only writes the PRF (a physical register), which *can* be discarded on a flush (return to free list), so loads can execute speculatively.
2. Because the ALU-only design has exactly 2 ALUs and 2 PRF write ports — a 1-to-1 match. No two writers ever compete for the same port. Adding a MUL creates a third writer, so now 3 FUs may finish simultaneously but only 2 ports exist; one must wait. This is when arbitration becomes necessary.
3. “Slow but correct” because a blocking LSU serializes all memory operations — a load can't start until the previous memory op finishes, even if they're independent. This wastes potential parallelism but never produces wrong results. A load-store

queue would allow multiple in-flight memory ops, out-of-order loads, and store-to-load forwarding (a load reading a not-yet-committed store's value), recovering parallelism at the cost of significant complexity.